

High performance computing systems

Lab 5

Dept. of Computer Architecture
Faculty of ETI
Gdansk University of Technology

Paweł Czarnul

updates: Robert Kałaska

For this exercise, study support for multithreading offered by MPI. This refers to the possibility of calling MPI functions from multiple threads started within a process.

Namely, an MPI implementation may support one of the following support levels for using threads:

1. `MPI_THREAD_SINGLE` – no support,
2. `MPI_THREAD_SERIALIZED` – threads are allowed to call MPI functions, but only one at a time,
3. `MPI_THREAD_FUNNELED` – only the thread that initialized MPI will call MPI functions,
4. `MPI_THREAD_MULTIPLE` – no restrictions.

Instead of `MPI_Init`, `MPI_Init_thread` should be called to initialize MPI and thread support. A program requests a certain level of thread support while MPI returns the level it supports.

Study the MPI specification for details.

The application presented here is an extended version of the program from lab1. That is, it computes pi in parallel using an old method from the 17th century:

$$\pi/4 = 1/1 - 1/3 + 1/5 - 1/7 + 1/9 \dots$$

A similar method to lab1 was adopted. However, in this case the program requests `MPI_THREAD_MULTIPLE` from MPI. Each process calculates subsum over its range depending on process rank. Subsum within process are calculated in parallel using threads (number of threads defined in `THREADNUM`). Then, successive elements of the aforementioned sum will be assigned to the threads of process 0, the threads of process 1, ..., the threads of process (n-1).

Note that you need to use an MPI implementation that supports `MPI_THREAD_MULTIPLE`

Compilation instructions:

```
mpicc -fopenmp sample.c -o sample
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <mpi.h>
```

```
#include <omp.h>
```

```
#define THREADNUM 8
```

```
#define RESULT 1
```

```
double precision = 10000000000;
```

```
double step;
```

```
int myrank, proccount;
```

```
void calculate (int rank)
```

```
{
```

```
    double pi_part = 0;
```

```
    double elem=0;
```

```
    int end = rank*step+step;
```

```
    int i=0;
```

```
    #pragma omp parallel for private(elem) reduction(+:pi_part)
```

```
    for(i=rank*step;i<end;i++) {
```

```
        if(i%2) {
```

```
            elem = (-1) * 1.0/((2*i)+1);
```

```
        } else {
```

```
            elem = 1.0/((2*i)+1);
```

```
        }
```

```
    pi_part = pi_part + elem;  
}
```

```
MPI_Send (&pi_part, 1, MPI_DOUBLE, 0, RESULT, MPI_COMM_WORLD);  
}
```

```
int main (int argc, char **argv)
```

```
{  
    omp_set_num_threads(THREADNUM);
```

```
    double pi_final = 0;
```

```
    int i;
```

```
    int threadsupport;
```

```
    MPI_Status status;
```

```
    // Initialize MPI
```

```
    MPI_Init_thread (&argc, &argv, MPI_THREAD_MULTIPLE, &threadsupport);
```

```
    if (threadsupport != MPI_THREAD_MULTIPLE)
```

```
    {
```

```
        printf ("\nThe implementation does not support MPI_THREAD_MULTIPLE, it supports  
level %d\n",
```

```
                threadsupport);
```

```
        MPI_Finalize();
```

```
        exit (-1);
```

```
    }
```

```
    // find out my rank
```

```
MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
```

```
// find out the number of processes in MPI_COMM_WORLD
```

```
MPI_Comm_size (MPI_COMM_WORLD, &proccount);
```

```
// now distribute the required precision
```

```
if (precision < proccount)
```

```
{
```

```
    printf("Precision smaller than the number of processes - try again.");
```

```
    MPI_Finalize ();
```

```
    return -1;
```

```
}
```

```
// initialize the step value
```

```
step = precision / proccount;
```

```
    calculate(myrank);
```

```
if (!myrank)
```

```
{
```

```
    // receive results from the threads
```

```
        double resulttemp;
```

```
        for (i = 0; i < proccount; i++)
```

```
        {
```

```
            MPI_Recv (&resulttemp, 1, MPI_DOUBLE, i, RESULT, MPI_COMM_WORLD,  
&status);
```

```
            printf ("\nReceived result %f for process %d\n",resulttemp, i);
```

```
    fflush (stdout);  
    pi_final += resulttemp;  
    }  
}
```

```
if (!myrank)  
{  
    pi_final *= 4;  
    printf ("\npi=%f\n", pi_final);  
}
```

```
// Shut down MPI  
MPI_Finalize ();  
return 0;  
}
```